ЛАБОРАТОРИЙН АЖЛЫН ГАРЫН АВЛАГА

Программ хангамжийн хөгжүүлэлтийн процесс

Программ хангамжийн инженерчлэл хөтөлбөр — 7-р улирал

16 лабораторийн алхам алхмаар заавар, бие даан хийх даалгавар

16 ЛАБОРАТОРИЙН ТОЙМ

Лаб	Долоо хоног	Сэдэв	Холбогдох CLO	Хугацаа
1	1	Хөгжүүлэлтийн орчин тохируулах, Git/GitHub суурь	CLO1	2 цаг
2	2	Хэрэглэгчийн түүх бичих, Backlog үүсгэх	CLO3	2 цаг
3	3	Scrum Simulation — Sprint Planning	CLO2	2 цаг
4	4	Kanban самбар зохион байгуулах, Cycle/Lead Time тооцоолол	CLO2	2 цаг
5	5	UML диаграм зурах, архитектурын эскиз бэлдэх	CLO3	2 цаг
6	6	Git Branching, Merge Conflict шийдвэрлэх, Code Review	CLO4	2 цаг
7	7	GitHub Actions ашиглан CI Pipeline байгуулах	CLO4	2 цаг
8а	8	Бие дааж хийх даалгавар 1 дээр зөвлөгөө авах семинар	CLO1–CLO3	2 цаг
8	9	Dockerfile бичих, Image бүтээх, ажиллуулах	CLO4	2 цаг
9	10	Minikube орчинд Deployment, Service үүсгэх	CLO4	2 цаг
10	11	Автомат тест бичих, CI Pipeline-д холбох	CLO4	2 цаг
11	12	Статик шинжилгээ, эмзэг байдал илрүүлэх	CLO4, CLO5	2 цаг
12	13	AI хэрэгсэл ашиглан код бичих, Review хийх	CLO5	2 цаг
13	14	Prometheus, Grafana ашиглан хяналтын самбар байгуулах	CLO4	2 цаг
14	15	DORA Metrics тооцоолол, Спринт Ретроспектив	CLO2, CLO6	2 цаг
16	16	Бие дааж хийх даалгавар 2 хамгаалалт, демо танилцуулга	CLO1–CLO6	2 цаг

ЛАБОРАТОРИ 1

Хөгжүүлэлтийн орчин тохируулах, Git/GitHub суурь

1-р долоо хоног (Лекц 1) · Хугацаа: 2 цаг · Холбогдох CLO: CLO1

Шаардлагатай хэрэгсэл: Git, GitHub, VS Code эсвэл өөр IDE, Терминал

Лабораторийн зорилго

1. Git, GitHub-ийн үндсэн орчныг бие даан тохируулах
2. Локал болон алсын (remote) repository-ийн ялгааг ойлгож, ашиглах
3. Анхны commit, push хийж, GitHub дээр код байршуулах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Git суулгах, тохируулах	Git-ийг компьютертоо суулгаад, `git config --global user.name "Нэр"` болон `git config --global user.email "имэйл"` командаар өөрийн мэдээллээ тохируулна.
2	GitHub эрхийн бүртгэл үүсгэх	github.com дээр бүртгэл үүсгэж, профайлаа бөглөнө. Хэрэв бүртгэлтэй бол нэвтэрнэ.
3	Шинэ репозитори үүсгэх	GitHub дээр "New repository" товч дарж, нэр өгөөд, README-тэйгээр үүсгэнэ.
4	Локал компьютер дээр clone хийх	`git clone <repository-url>` командаар репозиторийг локал компьютертоо хуулж авна.
5	Файл үүсгэж, өөрчлөлт хийх	Текст файл (жишээ нь `profile.md`) үүсгэж, өөрийн товч танилцуулгыг бичнэ.
6	Staging ба Commit хийх	`git add profile.md` командаар staging area-д нэмээд, `git commit -m "Add profile"` командаар баталгаажуулна.
7	Push хийх	`git push origin main` командаар GitHub дээрх алсын репозитори руу өөрчлөлтөө илгээнэ.
8	Баталгаажуулах	GitHub дээрх репозиторио хөтчөөрөө нээж, шинэ файл, commit түүх зөв харагдаж байгаа эсэхийг шалгана.

Бие дааж хийх даалгавар: Хувийн танилцуулга репозитори

Өөрийн нэрээр (жишээ нь `firstname-intro`) шинэ GitHub репозитори үүсгэ. Дотор нь дараах 2 файлыг үүсгэ: (1) `README.md` — өөрийн нэр, хөтөлбөр, энэ хичээлээс юу хүлээж байгаагаа 3-5 өгүүлбэрээр бичнэ; (2) `goals.md` — энэ улирлын турш эзэмшихийг зорьж буй 3 ур чадвар жагсаана. Хамгийн багадаа 3 тусдаа commit хийж, GitHub руу push хийнэ.

Хүлээлгэж өгөх зүйл (deliverable): *Репозиторийн холбоос (URL)-ыг лабораторийн тайландаа хавсаргана.*

ЛАБОРАТОРИ 2

Хэрэглэгчийн түүх бичих, Backlog үүсгэх

2-р долоо хоног (Лекц 2) · Хугацаа: 2 цаг · Холбогдох CLO: CLO3

Шаардлагатай хэрэгсэл: Trello эсвэл Jira (үнэгүй хувилбар), Текст боловсруулагч

Лабораторийн зорилго

1. INVEST шалгуурт нийцсэн хэрэглэгчийн түүх (user story) бичих
2. Given-When-Then форматаар acceptance criteria тодорхойлох
3. Product Backlog-ийг Trello/Jira дээр бодитоор үүсгэж, эрэмбэлэх

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Кейс сонгох	Багш өгсөн 3 кейсийн нэгийг (жишээ нь: онлайн номын сан, хүнсний хүргэлтийн апп, хувийн санхүүгийн трекер) сонгоно.
2	Хэрэглэгчийн дүрийг тодорхойлох	Тухайн системийг ашиглах 2-3 өөр төрлийн хэрэглэгчийн дүр (persona) тодорхойлно, жишээ нь "шинэ хэрэглэгч", "админ".
3	User story бичих	Дүр бүрд "[Хэн] нь [юу хийхийг] хүсдэг, учир нь [шалтгаан]" форматаар хамгийн багадаа 8 user story бичнэ.
4	INVEST шалгууруар шалгах	Бичсэн story бүрийг INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable) шалгууруар нэг бүрчлэн шалгаж, шаардлагатай бол засварлана.
5	Acceptance Criteria нэмэх	Story бүрд хамгийн багадаа 1 Given-When-Then форматтай acceptance criteria бичнэ.
6	Trello/Jira дээр Backlog үүсгэх	Trello дээр шинэ самбар үүсгэж, "Product Backlog" багана нэмээд, story бүрийг тусдаа карт болгон оруулна.
7	Тэргүүлэх ач холбогдлоор эрэмбэлэх	Хэрэглэгчид хамгийн их үнэ цэнэ өгөх story-г самбарын дээд хэсэгт байрлуулж эрэмбэлнэ.

Бие дааж хийх даалгавар: Онлайн номын сангийн Backlog

"Онлайн номын сангийн систем" кейс дээр: (1) 2 хэрэглэгчийн дүр (жишээ нь уншигч, номын санч) тодорхойлж, (2) дүр бүрт 4-с доошгүй, нийт 8-с доошгүй user story бич, (3) story тус бүрд Given-When-Then acceptance criteria нэм, (4) бүгдийг Trello самбар дээр Backlog болгон үүсгэж, ач холбогдлоор нь эрэмбэл.

Хүлээлгэж өгөх зүйл (deliverable): *Trello самбарын холбоос эсвэл screenshot-той PDF.*

ЛАБОРАТОРИ 3

Scrum Simulation — Sprint Planning

3-р долоо хоног (Лекц 3) · Хугацаа: 2 цаг · Холбогдох CLO: CLO2

Шаардлагатай хэрэгсэл: Лаб 2-ын Backlog, Цаас/цахим самбар (Trello)

Лабораторийн зорилго

1. Scrum-ийн 3 үүргийг (Product Owner, Scrum Master, Dev баг) багаараа хуваарилж гүйцэтгэх
2. Product Backlog-оос Sprint Backlog гаргах дадлага хийх
3. Story point ашиглан Planning Poker техникээр тооцоолол хийх

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Баг бүрдүүлэх	3-4 хүний баг үүсгэж, дотроос нь 1 Product Owner, 1 Scrum Master, үлдсэн нь Development баг гэж үүрэг хуваарилна.
2	Backlog-оо бэлтгэх	Лаб 2 дээр бичсэн Product Backlog-оо (эсвэл багшийн өгсөн жишиг backlog-ыг) бэлэн болгоно.
3	Багийн хүчин чадлыг тодорхойлох	2 долоо хоногийн sprint-д таны баг ойролцоогоор хэдэн story point гүйцэтгэж чадах вэ гэдгийг тохиролцоно (жишээ нь 20 point).
4	Planning Poker хийх	Backlog item бүрийг Fibonacci карт (1,2,3,5,8,13) ашиглан баг гишүүн бүр нууцаар онооно, дараа нь нээж, зөрүүтэй бол хэлэлцэж эцсийн тоог тохирно.
5	Sprint Backlog сонгох	Тооцоолсон нийт point хүчин чадлаас хэтрэхгүйгээр, хамгийн өндөр ач холбогдолтой story-г Sprint Backlog болгон сонгоно.
6	Даалгавар болгон задлах	Sprint Backlog дэх story бүрийг 2-4 тодорхой техникийн даалгавар (task) болгон задална.
7	Sprint Goal бичих	Энэ sprint-ийн ерөнхий зорилгыг 1 өгүүлбэрээр тодорхойлж бичнэ.

Бие дааж хийх даалгавар: Sprint Backlog баримт бичиг

Багаараа сонгосон Sprint Backlog-оо баримтжуулна: (1) Sprint Goal, (2) сонгосон story-уудын жагсаалт, story point-той, (3) story тус бүрийн задарсан даалгаврууд, (4) багийн гишүүн тус бүрд хуваарилсан ажил.

Хүлээлгэж өгөх зүйл (deliverable): *1 хуудас баримт бичиг (Word/Markdown) эсвэл Trello самбарын screenshot.*

ЛАБОРАТОРИ 4

Kanban самбар зохион байгуулах, Cycle/Lead Time тооцоолол

4-р долоо хоног (Лекц 4) · Хугацаа: 2 цаг · Холбогдох CLO: CLO2

Шаардлагатай хэрэгсэл: Trello эсвэл Excel, Лаб 2-3-ын backlog

Лабораторийн зорилго

1. Kanban самбар бодитоор байгуулж, WIP хязгаарлалт тохируулах
2. Cycle Time, Lead Time-ийг бодит өгөгдлөөс тооцоолох
3. Cumulative Flow Diagram-ийн үндсэн зарчмыг ойлгож ашиглах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Kanban самбар үүсгэх	Trello дээр шинэ самбар үүсгэж, To Do, In Progress, Review, Done гэсэн 4 багана нэмнэ.
2	WIP хязгаарлалт тохируулах	In Progress болон Review баганад тус бүр 2-3 картын дээд хязгаар тохируулна (Trello card limit тэмдэглэгээ ашиглах).
3	Card үүсгэх	Лаб 2-3-ын backlog-оос сонгосон 8-10 ажлыг To Do баганад карт болгон оруулна.
4	Симуляц ажиллуулах	Багш өгсөн хугацааны симуляцийн дагуу карт бүрийг To Do -> In Progress -> Review -> Done чиглэлд шилжүүлж, шилжсэн огноог тэмдэглэнэ.
5	Өгөгдөл цуглуулах	Карт бүрийн "эхэлсэн" (In Progress болсон), "дууссан" (Done болсон) огноог хүснэгтэд бичнэ.
6	Cycle Time тооцоолох	Карт бүрийн Cycle Time = Done болсон огноо − In Progress болсон огноо гэдгийг тооцоолж, дундажийг гаргана.
7	CFD зурах	Excel-д өдөр тутмын багана бүрийн картын тоог хүснэгтэд бичиж, давхарласан (stacked area) график зурна.

Бие дааж хийх даалгавар: Cycle Time дүн шинжилгээ

Симуляцийн үр дүнгээс: (1) карт бүрийн Cycle Time-ийг тооцоолсон хүснэгт гарга, (2) дундаж Cycle Time-ийг тооцоол, (3) хамгийн удаан явсан картыг тодорхойлж, яагаад удсан бэ гэдгийг (bottleneck) тайлбарла, (4) WIP хязгаарлалтгүй бол илүү хурдан байх байсан уу гэдгээ 2-3 өгүүлбэрээр дүгнэ.

Хүлээлгэж өгөх зүйл (deliverable): *Excel хүснэгт + богино дүгнэлт (≠ хуудас).*

ЛАБОРАТОРИ 5

UML диаграм зурах, архитектурын загвар бэлтгэх

5-р долоо хоног (Лекц 5) · Хугацаа: 2 цаг · Холбогдох CLO: CLO3

Шаардлагатай хэрэгсэл: draw.io (diagrams.net) эсвэл Lucidchart, Лаб 2-ын шаардлага

Лабораторийн зорилго

1. Class Diagram, Use Case Diagram, Sequence Diagram зурах дадлага хийх
2. Системийн архитектурын эскизийг (Layered эсвэл Client-Server) зурж, үндэслэх

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Хэрэгсэл нээх	draw.io (diagrams.net)-г хөтчөөр нээж, шинэ, хоосон diagram үүсгэнэ.
2	Use Case Diagram зурах	Лаб 2-ын хэрэглэгчийн дүрүүдийг "actor" болгож, тэдний хийж чадах үйлдлүүдийг (use case)-ыг зурж холбоно.
3	Class Diagram зурах	Системийн үндсэн 4-5 объект (жишээ нь User, Book, Loan) сонгож, тэдгээрийн attribute, method-ийг класс хайрцаг болгон зурж, хоорондын харилцааг (association, inheritance) шугамаар холбоно.
4	Sequence Diagram зурах	Нэг чухал урсгал (жишээ нь "хэрэглэгч ном захиалах") сонгож, хэрэглэгч -> систем -> өгөгдлийн сан хоорондын дараалсан харилцан үйлдлийг цаг хугацааны дарааллаар зурна.
5	Архитектурын эскиз зурах	Системээ Layered (Presentation-Business-Data) эсвэл Client-Server бүтцээр өндөр түвшинд дүрсэлж зурна.
6	Үндэслэл бичих	Яагаад энэ архитектурыг сонгосноо (багийн хэмжээ, тэлэлтийн шаардлага гэх мэт үндэслэлээр) 3-4 өгүүлбэрээр бичнэ.

Бие дааж хийх даалгавар: Бүрэн архитектурын багц баримт бичиг

Онлайн номын сангийн кейс дээр: (1) Use Case Diagram (2 дүр, 5+ use case), (2) Class Diagram (4-5 класс, attribute/method-той), (3) Sequence Diagram (1 гол урсгал), (4) Архитектурын эскиз + 3-4 өгүүлбэр үндэслэл — бүгдийг нэг PDF болгон экспортол.

Хүлээлгэж өгөх зүйл (deliverable): *PDF файл (4 диаграм + үндэслэл).*

ЛАБОРАТОРИ 6

Git Branching, Merge Conflict шийдвэрлэх, Code Review

6-р долоо хоног (Лекц 6) · Хугацаа: 2 цаг · Холбогдох CLO: CLO4

Шаардлагатай хэрэгсэл: Git, GitHub (Pull Request)

Лабораторийн зорилго

1. Feature branch үүсгэж, ажиллаж, main руу нэгтгэх дадлага хийх
2. Санаатайгаар merge conflict үүсгэж, шийдвэрлэх
3. Pull Request үүсгэж, хосоороо code review хийх

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Хос бүрдүүлэх	2 хүний хос болж, аль аль нь Лаб 1-ийн репозитори эсвэл багшийн өгсөн жишиг репозитори дээр хамтран ажиллана.
2	Feature branch үүсгэх	`git checkout -b feature/greeting` командаар шинэ branch үүсгэнэ.
3	Өөрчлөлт хийх, commit хийх	Тухайн branch дээр файлд өөрчлөлт хийж commit хийнэ.
4	Pull Request үүсгэх	GitHub дээр "New Pull Request" товч дарж, feature branch-аа main руу нэгтгэхийг хүссэн PR үүсгэнэ, тайлбар бичнэ.
5	Хосоороо conflict үүсгэх	Хоёулаа ижил файлын ижил мөрийг өөр өөрөөр өөрчилж, тус тусдаа branch дээрээ commit, push хийнэ.
6	Merge Conflict шийдвэрлэх	Хоёр дахь хүн main руу merge хийхийг оролдоход conflict гарна — тэмдэглэгээг олж, аль хувилбарыг авахаа хамтран шийдэж, файлыг зассаны дараа commit хийнэ.
7	Code Review хийх	Хос нэг нэгнийхээ PR дээр хамгийн багадаа 2 тайлбар (comment) бичиж, сайжруулах санал өгнө.
8	Merge хийх	Review дууссаны дараа PR-ийг main руу merge хийнэ.

Бие дааж хийх даалгавар: Branch-Conflict-Review бүрэн мөчлөг

Хосоороо: (1) 2 тусдаа feature branch үүсгэж, (2) санаатайгаар merge conflict үүсгэж шийдвэрлэ, (3) хоёулаа нэг нэгнийхээ PR-д хамгийн багадаа 2 бодит, ашигтай тайлбар бичиж review хийнэ, (4) эцэст нь бүх branch-ийг main руу амжилттай merge хийнэ.

Хүлээлгэж өгөх зүйл (deliverable): *GitHub репозиторийн холбоос — commit мүүх, PR, comment-үүд харагдана.*

ЛАБОРАТОРИ 7

GitHub Actions ашиглан CI Pipeline байгуулах

7-р долоо хоног (Лекц 7) · Хугацаа: 2 цаг · Холбогдох CLO: CLO4

Шаардлагатай хэрэгсэл: GitHub Actions, Node.js эсвэл Python жишиг төсөл

Лабораторийн зорилго

1. GitHub Actions workflow файл бие даан бичих
2. Push, Pull Request trigger дээр автомат build/test ажиллуулах
3. CI pipeline-ийн амжилттай/амжилтгүй үр дүнг унших, тайлбарлах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Жишиг төсөл бэлдэх	Багшийн өгсөн энгийн Node.js эсвэл Python төслийг (үндсэн тестүүдтэй) өөрийн репозитори руу clone хийнэ.
2	Workflow хавтас үүсгэх	Репозиторийн үндсэн хавтсанд `.github/workflows/` хавтас үүсгэнэ.
3	ci.yml файл бичих	`ci.yml` файл үүсгэж, `name`, `on: [push, pull_request]`, `jobs` хэсгийг бичиж эхэлнэ.
4	Checkout, Setup алхам нэмэх	`actions/checkout@v4`, дараа нь орчны тохиргоо (жишээ нь `actions/setup-node@v4`) алхмыг нэмнэ.
5	Install, Test алхам нэмэх	Хамаарал суулгах (`npm install` эсвэл `pip install`), дараа нь тест ажиллуулах (`npm test` эсвэл `pytest`) алхмыг нэмнэ.
6	Push хийж, ажиллуулах	Файлаа commit, push хийж, GitHub дээрх "Actions" таб руу орж, workflow автоматаар эхэлж байгааг ажиглана.
7	Амжилттай үр дүн шалгах	Бүх алхам ногоон (амжилттай) болсныг баталгаажуулна.
8	Санаатайгаар унагаах	Тестийг санаатайгаар эвдэж (жишээ нь буруу утга оруулж) дахин push хийж, CI улаан болж, алдааны дэлгэрэнгүй лог хэрхэн харагддагийг ажиглана.

Бие дааж хийх даалгавар: Бүрэн CI Pipeline

Өөрийн репозиторид: (1) checkout, setup, install, test 4 алхамтай CI workflow бич, (2) push/PR trigger-тэй байлга, (3) амжилттай ажиллаж байгааг screenshot-оор баталгаажуул, (4) дараа нь санаатайгаар 1 тест унагааж, амжилтгүй build-ийн screenshot бас авч, юу болсныг тайлбарла.

Хүлээлгэж өгөх зүйл (deliverable): *Репозиторийн холбоос + 2 screenshot (амжилттай, амжилтгүй) + богино тайлбар.*

ЛАБОРАТОРИ 8а

Бие дааж хийх даалгавар 1 дээр зөвлөгөө авах семинар

8-р долоо хоног (Лекц 8) · Хугацаа: 2 цаг · Холбогдох CLO: CLO1–CLO3

Шаардлагатай хэрэгсэл: Багийн Бие дааж хийх даалгавар 1-ийн ноорог

Лабораторийн зорилго

1. Бие дааж хийх даалгавар 1-ийн одоогийн ахицаа танилцуулж, зөвлөгөө авах
2. Бусад багийн ажлаас суралцах, харьцуулах
3. Хамгаалалтын өмнөх сүүлийн засварын жагсаалт гаргах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Багаараа бэлдэх	Ирэхийн өмнө багаараа одоогийн ахицаа (SDLC үндэслэл, шаардлага, backlog, архитектур) нэгтгэж, ямар асуулт, бэрхшээлтэй байгаагаа тодорхойлно.
2	5 минутын танилцуулга	Баг бүр ээлжлэн 5 минутын дотор одоогийн ажлаа ангид товч танилцуулна.
3	Багшийн санал, засвар	Багш танилцуулга бүрт шууд санал, засах зүйлийг өгнө.
4	Багийн хоорондын feedback	Бусад багийн 1-2 гишүүн танилцуулга бүрд 1 эерэг, 1 сайжруулах санал өгнө.
5	Ажлын жагсаалт гаргах	Багш, ангийнхны feedback дээр үндэслэн, хамгаалалтаас өмнө хийх ёстой тодорхой ажлын жагсаалт (checklist) гаргана.

Бие дааж хийх даалгавар: Засварын жагсаалт (Action Items)

Семинарын үеэр авсан бүх feedback-ээ нэгтгэж, багаараа хамгаалалтаас өмнө хийх тодорхой 5-8 ажлын жагсаалт (жишээ нь "Архитектурын диаграмд Database давхарга нэмэх", "Story 3-т acceptance criteria дутуу — нэмэх") гарга.

Хүлээлгэж өгөх зүйл (deliverable): *Ажлын жагсаалт (checklist) баримт бичиг.*

ЛАБОРАТОРИ 8

Dockerfile бичих, Image бүтээх, ажиллуулах

9-р долоо хоног (Лекц 9) · Хугацаа: 2 цаг · Холбогдох CLO: CLO4

Шаардлагатай хэрэгсэл: Docker Desktop, Жишиг веб аппликейшн (Node.js/Python)

Лабораторийн зорилго

1. Dockerfile бие даан бичиж, layer caching-ийн зарчмыг ашиглах
2. Docker image бүтээж, container ажиллуулах
3. Docker Compose ашиглан олон сервис зэрэг ажиллуулах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Docker суулгасан эсэхийг шалгах	`docker --version` командаар Docker Desktop зөв суусан эсэхийг шалгана.
2	Жишиг апп бэлдэх	Багшийн өгсөн энгийн веб апп (жишээ нь Express "Hello World" сервер)-ыг локал руугаа татаж авна.
3	Dockerfile бичих	FROM, WORKDIR, COPY package.json, RUN npm install, COPY . ., CMD зааврыг зөв дараалалтай бичнэ (layer caching-ийг санаж).
4	Image бүтээх	`docker build -t myapp:1.0 .` командаар image бүтээнэ.
5	Container ажиллуулах	`docker run -p 3000:3000 myapp:1.0` командаар container ажиллуулж, browser дээр localhost:3000 нээж шалгана.
6	Логуудыг харах	`docker logs <container-id>` командаар container-ийн консол гаралтыг харна.
7	Docker Compose нэмэх	Хэрэв апп өгөгдлийн сан ашигладаг бол `docker-compose.yml` бичиж, апп + database хамт `docker compose up`-аар ажиллуулна.
8	Layer caching ажиглах	Кодны файлдаа жижиг өөрчлөлт хийгээд дахин `docker build` хийж, аль layer кэшнээс ашиглагдсаныг терминал дээр ажиглана.

Бие дааж хийх даалгавар: Контейнержуулсан апп

Өөрийн Бие дааж хийх даалгавар 2-ын (эсвэл жишиг) төслийг: (1) зөв дарааллтай Dockerfile бичиж контейнержуул, (2) image бүтээж, локал ажиллуулж screenshot ав, (3) кодны жижиг өөрчлөлт хийгээд дахин build хийж, layer caching ажилласан эсэхийг terminal output-оор баталгаажуул.

Хүлээлгэж өгөх зүйл (deliverable): *Dockerfile + 2 screenshot (ажиллаж буй апп, build output).*

ЛАБОРАТОРИ 9

Minikube орчинд Deployment, Service үүсгэх

10-р долоо хоног (Лекц 10) · Хугацаа: 2 цаг · Холбогдох CLO: CLO4

Шаардлагатай хэрэгсэл: Minikube, kubectl

Лабораторийн зорилго

1. Minikube локал Kubernetes cluster эхлүүлэх
2. Deployment, Service YAML файл бие даан бичиж хэрэгжүүлэх
3. kubectl командаар cluster-ийн байдлыг шалгах, оношлох

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Minikube эхлүүлэх	`minikube start` командаар локал Kubernetes cluster-ээ эхлүүлнэ.
2	Cluster байдлыг шалгах	`kubectl get nodes` командаар cluster ажиллаж байгааг баталгаажуулна.
3	Deployment YAML бичих	Лаб 8-д бүтээсэн Docker image-аа ашиглан `deployment.yaml` файл бичиж, `replicas: 3` тохируулна.
4	Deployment хэрэгжүүлэх	`kubectl apply -f deployment.yaml` командаар Deployment-ээ cluster-т илгээнэ.
5	Pod-уудыг шалгах	`kubectl get pods` командаар яг 3 Pod ажиллаж байгааг харна.
6	Service YAML бичих	`service.yaml` файл бичиж, Deployment-ээ гадагшаа нээх Service тодорхойлно.
7	Self-healing туршиж үзэх	`kubectl delete pod <pod-name>` командаар нэг Pod-ийг санаатайгаар устгаж, Kubernetes автоматаар шинээр үүсгэж байгааг ажиглана.
8	Аппликейшн руу хандах	`minikube service <service-name>` командаар браузераар ажиллаж буй апп руу хандаж шалгана.

Бие дааж хийх даалгавар: Deployment + Self-Healing баталгаажуулалт

(1) Өөрийн Docker image ашиглан 3 replica-тай Deployment, харгалзах Service YAML бич, (2) `kubectl apply` хийж ажиллуулж screenshot ав, (3) нэг Pod-ийг санаатайгаар устгаад, Kubernetes автоматаар дахин үүсгэсэн screenshot ав.

Хүлээлгэж өгөх зүйл (deliverable): *deployment.yaml + service.yaml + 2 screenshot.*

ЛАБОРАТОРИ 10

Автомат тест бичих, CI Pipeline-д холбох

11-р долоо хоног (Лекц 11) · Хугацаа: 2 цаг · Холбогдох CLO: CLO4

Шаардлагатай хэрэгсэл: Jest (JavaScript) эсвэл JUnit/PyTest, Лаб 7-ийн CI pipeline

Лабораторийн зорилго

1. AAA (Arrange-Act-Assert) зарчмаар unit тест бичих
2. Mock ашиглан гадаад хамаарлыг тусгаарлах
3. Бичсэн тестээ Лаб 7-ийн CI pipeline-д нэгтгэх

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Тестийн санг суулгах	`npm install --save-dev jest` (эсвэл `pip install pytest`) командаар тестийн сан суулгана.
2	Тестлэх функц сонгох	Багшийн өгсөн жишиг код дотроос (жишээ нь `calculateTotal()`) тестлэх функц сонгоно.
3	Эхний unit тест бичих	AAA зарчмаар: Arrange (жишиг өгөгдөл бэлдэх) -> Act (функцийг дуудах) -> Assert (үр дүнг шалгах) гэсэн бүтэцтэй тест бичнэ.
4	Edge case тест нэмэх	Хоосон утга, сөрөг тоо, маш том тоо зэрэг хилийн тохиолдлуудад зориулсан нэмэлт 2-3 тест бичнэ.
5	Mock ашиглах	Хэрэв функц гадаад API/өгөгдлийн сан дуудаж байвал, mock объект ашиглан тэдгээрийг орлуулж, зөвхөн логикийг тусгаарлан тестлэнэ.
6	Локал ажиллуулах	`npm test` (эсвэл `pytest`) командаар бүх тестээ локал ажиллуулж, бүгд ногоон (амжилттай) байгааг шалгана.
7	CI-д нэгтгэх	Лаб 7 дээр бичсэн `ci.yml`-д тест ажиллуулах алхам аль хэдийн байгаа эсэхийг шалгаж, шаардлагатай бол шинэчилнэ.
8	Санаатайгаар унагаах	Кодондоо санаатай алдаа оруулж push хийж, CI дээр тест унаж, PR улаан болохыг ажиглана.

Бие дааж хийх даалгавар: Тестийн иж бүрдэл

Сонгосон функцэд: (1) хэвийн тохиолдлын 2 unit тест, (2) хилийн (edge case) 2 тест, (3) шаардлагатай бол mock ашигласан 1 тест бич — нийт 5 тест. Бүгдийг локал ажиллуулж, дараа нь CI pipeline дээр амжилттай явуулж screenshot ав.

Хүлээлгэж өгөх зүйл (deliverable): *Тестийн файл + CI амжилттай ажилласан screenshot.*

ЛАБОРАТОРИ 11

Статик шинжилгээ, эмзэг байдал илрүүлэх

12-р долоо хоног (Лекц 12) · Хугацаа: 2 цаг · Холбогдох CLO: CLO4, CLO5

Шаардлагатай хэрэгсэл: ESLint/Pylint, npm audit эсвэл SonarQube (community)

Лабораторийн зорилго

1. Linter тохируулж, кодны стилийн алдааг автоматаар илрүүлэх
2. Dependency scanning ашиглан гуравдагч party сангийн эмзэг байдлыг олох
3. Илэрсэн асуудлыг ач холбогдлоор нь ангилж, засах төлөвлөгөө гаргах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Linter суулгах	`npm install --save-dev eslint` (эсвэл `pip install pylint`) командаар суулгана.
2	Тохиргоо үүсгэх	`.eslintrc` (эсвэл `.pylintrc`) файл үүсгэж, суурь дүрмийн багц сонгоно.
3	Linter ажиллуулах	`npx eslint .` командаар өөрийн кодон дээр ажиллуулж, гарсан анхааруулга, алдааны жагсаалтыг харна.
4	Асуудлыг засах	Илэрсэн 5-10 асуудлыг (жишээ нь ашиглагдаагүй хувьсагч, тохирохгүй нэршил) нэг бүрчлэн засна.
5	Dependency scan хийх	`npm audit` (эсвэл GitHub Dependabot тохиргоо) командаар ашиглаж буй сангуудын мэдэгдэж буй эмзэг байдлыг шалгана.
6	Үр дүнг ангилах	Илэрсэн эмзэг байдлыг Critical, High, Medium, Low гэсэн ангилалд ялгаж, аль нэрвэгдсэн санг шинэчлэх шаардлагатайг тодорхойлно.
7	CI-д нэгтгэх	Лаб 7-ийн CI workflow-д `npx eslint .` алхмыг нэмж, код push бүрд автоматаар ажиллахаар тохируулна.

Бие дааж хийх даалгавар: Кодны чанар, аюулгүй байдлын тайлан

Өөрийн төсөл дээр: (1) linter ажиллуулж, илэрсэн бүх асуудлыг засах, (2) `npm audit` ажиллуулж эмзэг сангуудыг тодорхойлох, (3) Critical/High эрсдэлтэй сангуудыг шинэчлэх (эсвэл шинэчлэх боломжгүй бол шалтгаанаа тайлбарлах), (4) linter шалгалтыг CI pipeline-д нэгтгэх.

Хүлээлгэж өгөх зүйл (deliverable): *"Өмнө/дараа" linter гаралтын харьцуулалт + `npm audit` тайлан + шинэчилсэн CI workflow.*

ЛАБОРАТОРИ 12

AI хэрэгсэл ашиглан код бичих, Review хийх

13-р долоо хоног (Лекц 13) · Хугацаа: 2 цаг · Холбогдох CLO: CLO5

Шаардлагатай хэрэгсэл: GitHub Copilot / Claude Code / өөр AI код туслах

Лабораторийн зорилго

1. Тодорхой, контексттэй prompt бичих дадлага хийх
2. AI-ийн санал болгосон кодыг шүүмжлэлтэй review хийх
3. AI ашигласан тохиолдлоо ил тод баримтжуулах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	AI хэрэгслээ бэлдэх	GitHub Copilot эсвэл Claude Code зэрэг AI хэрэгслээ идэвхжүүлнэ (багшийн зааврын дагуу).
2	Даалгавраа сонгох	Багшийн өгсөн жижиг функц (жишээ нь "нэвтрэлтийн validation") бичих даалгаврыг сонгоно.
3	Муу prompt-оор эхлэх	Эхлээд санаатайгаар тодорхойгүй, ерөнхий prompt өгч, AI ямар хариу өгдгийг ажиглана.
4	Сайн prompt бичих	Технологи, шаардлага, аюулгүй байдлын нөхцлийг тодорхой зааж, дэлгэрэнгүй prompt дахин өгнө.
5	AI-ийн саналыг унших	AI-ийн буцаасан кодыг шууд хуулахгүйгээр, эхлээд бүрэн уншиж, логикийг ойлгоно.
6	Review хийх	Лекц 13-т судалсан шалгууруар (логик зөв үү, аюулгүй юу, стилд нийцэж байна уу) кодыг шалгаж, асуудал олно.
7	Тестээр баталгаажуулах	AI-ийн санал болгосон кодод Лаб 10-т сурсан аргаар unit тест бичиж ажиллуулна.
8	Ашиглалтын тэмдэглэл хөтлөх	AI-д өгсөн prompt, гарсан анхны хариу, өөрийн хийсэн засвар зэргийг тэмдэглэж баримтжуулна.

Бие дааж хийх даалгавар: AI-Review тайлан

(1) Ерөнхий, тодорхойгүй prompt болон тодорхой, дэлгэрэнгүй prompt хоёрыг харьцуулж, AI-ийн хариу хэрхэн ялгаатай байсныг бич, (2) эцсийн AI-ийн санал болгосон кодыг review хийж, олдсон 2-3 асуудлыг тэмдэглэ (эсвэл "асуудал олдоогүй, учир нь..." гэж үндэслэ), (3) кодыг тестээр баталгаажуулж, эцсийн, засварласан хувилбараа хавсарга.

Хүлээлгэж өгөх зүйл (deliverable): *Prompt харьцуулалт + review тэмдэглэл + эцсийн код, тест.*

ЛАБОРАТОРИ 13

Prometheus, Grafana ашиглан хяналтын самбар байгуулах

14-р долоо хоног (Лекц 14) · Хугацаа: 2 цаг · Холбогдох CLO: CLO4

Шаардлагатай хэрэгсэл: Prometheus, Grafana, Docker Compose

Лабораторийн зорилго

1. Prometheus, Grafana-г Docker Compose ашиглан локал ажиллуулах
2. Аппликейшнаас metrics цуглуулж, Prometheus-т холбох
3. Grafana dashboard бүтээж, metrics-ийг дүрслэн харуулах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Docker Compose бэлдэх	Багшийн өгсөн `docker-compose.yml`-д Prometheus, Grafana сервисийг нэмнэ.
2	Metrics endpoint нэмэх	Лаб 8-ын аппликейшндаа `/metrics` гэдэг endpoint нэмж, суурь metrics (жишээ нь хүсэлтийн тоо) экспортлоно.
3	Prometheus тохиргоо	`prometheus.yml` файлд аппликейшны `/metrics` endpoint-ийг "scrape target" болгон бүртгэнэ.
4	Бүх сервисийг ажиллуулах	`docker compose up` командаар апп, Prometheus, Grafana-г нэг дор ажиллуулна.
5	Prometheus дээр шалгах	localhost:9090 (Prometheus UI) нээж, аппликейшны metrics зөв цуглуулагдаж байгааг харна.
6	Grafana холбох	localhost:3001 (Grafana) нээж, Prometheus-ийг өгөгдлийн эх сурвалж (data source) болгон нэмнэ.
7	Dashboard үүсгэх	Шинэ dashboard үүсгэж, хүсэлтийн тоо, хариу үйлдлийн хугацаа зэрэг 2-3 panel (график) нэмнэ.
8	Alert тохируулах	Нэг metric-т threshold тохируулж, хэтэрвэл дохиолол өгдөг энгийн alert rule үүсгэнэ.

Бие дааж хийх даалгавар: Ажиллагаатай хяналтын самбар

(1) Аппликейшнаасаа хамгийн багадаа 2 өөр metric (жишээ нь request count, response time) цуглуулж Prometheus-т холбо, (2) Grafana дээр 3-с доошгүй panel-тай dashboard бүтээ, (3) хиймэл ачаалал үүсгэж (жишээ нь хэдэн хүсэлт дараалан илгээх), dashboard дээр өөрчлөлт real-time харагдаж байгааг screenshot-оор баталгаажуул.

Хүлээлгэж өгөх зүйл (deliverable): *docker-compose.yml + dashboard screenshot + богино майлбар.*

ЛАБОРАТОРИ 14

DORA Metrics тооцоолол, Спринт Ретроспектив

15-р долоо хоног (Лекц 15) · Хугацаа: 2 цаг · Холбогдох CLO: CLO2, CLO6

Шаардлагатай хэрэгсэл: Excel эсвэл Google Sheets, Лаб 1-13-ын түүхэн өгөгдөл

Лабораторийн зорилго

1. DORA-ийн 4 үзүүлэлтийг бодит/жишиг өгөгдлөөс тооцоолох
2. Гүйцэтгэлийн ангилал (Elite/High/Medium/Low)-д харьцуулах
3. Бодит Sprint Retrospective хийж, сайжруулах төлөвлөгөө гаргах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Өгөгдөл цуглуулах	Багшийн өгсөн жишиг өгөгдлийн багц (10-15 deploy-ийн огноо, амжилттай/амжилтгүй тэмдэглэгээ, засварын хугацаа)-ыг нээнэ.
2	Deployment Frequency тооцоолох	Тухайн хугацаанд хэдэн удаа deploy хийсэн бэ гэдгийг тоолж, дундаж давтамжийг (өдөрт/долоо хоногт хэдэн удаа) тооцоолно.
3	Lead Time тооцоолох	Commit хийсэн огноо, deploy хийсэн огноог харьцуулж, дундаж Lead Time-ийг цагаар тооцоолно.
4	Change Failure Rate тооцоолох	Нийт deploy-ийн хэдэн хувь нь "амжилтгүй" (rollback шаардсан) тэмдэглэгээтэй байгааг хувиар тооцоолно.
5	Time to Restore тооцоолох	Амжилтгүй deploy бүрийн "илэрсэн" ба "засагдсан" огноог харьцуулж, дундаж сэргээх хугацааг тооцоолно.
6	Ангилалтай харьцуулах	Тооцоолсон 4 үзүүлэлтээ Elite/High/Medium/Low ангиллын хүснэгттэй харьцуулж, өөрийн (жишиг) багийн түвшинг тодорхойлно.
7	Sprint Retrospective хийх	Багаараа Лаб 1-13-ын туршлагаа эргэн харж, "юу сайн болсон, юу сайжруулах шаардлагатай, дараагийн удаа юуг өөрчлөх вэ" гэсэн 3 асуултад хариулна.
8	Action Items бичих	Retrospective-ийн үр дүнд, дараагийн (Бие дааж хийх даалгавар 2-т хэрэглэх) 3-5 тодорхой сайжруулалтын алхам бичнэ.

Бие дааж хийх даалгавар: DORA тайлан + Ретроспектив

(1) Өгөгдсөн жишиг өгөгдлөөс DORA-ийн 4 үзүүлэлтийг тооцоолж хүснэгт гарга, (2) багийн гүйцэтгэлийг Elite/High/Medium/Low ангилалд харьцуулж дүгнэ, (3) багаараа бодит Retrospective хийж, 3 асуултын хариулт + 3-5 action item бичсэн баримт бичиг бэлд.

Хүлээлгэж өгөх зүйл (deliverable): *DORA тооцооллын хүснэгт + Retrospective баримт бичиг.*

ЛАБОРАТОРИ 16

Бие дааж хийх даалгавар 2 хамгаалалт, демо танилцуулга

16-р долоо хоног (Лекц 16) · Хугацаа: 2 цаг · Холбогдох CLO: CLO1–CLO6

Шаардлагатай хэрэгсэл: Багийн бэлэн Бие дааж хийх даалгавар 2 (ажиллагаатай систем, CI/CD, баримтжуулалт)

Лабораторийн зорилго

1. Бүтэн улирлын турш хөгжүүлсэн төслөө бүрэн, тодорхой танилцуулах
2. CI/CD pipeline, тест, AI ашиглалтаа амьд демонстрацаар харуулах
3. Бусад багийн ажлыг үнэлэх, харилцан суралцах

Алхам алхмаар гүйцэтгэх заавар

№	Алхам	Тайлбар
1	Демогоо бэлдэх	Танилцуулгынхаа өмнө системээ ажиллуулж, CI/CD pipeline амжилттай ажиллаж байгааг баталгаажуулна.
2	Танилцуулгын бүтэц бэлдэх	5-7 минутын танилцуулгаа бэлдэнэ: (1) асуудал, шийдэл, (2) архитектур, (3) CI/CD demo, (4) AI ашигласан тохиолдол, (5) сурсан зүйл.
3	Ээлжлэн танилцуулах	Баг бүр ээлжлэн танилцуулж, амьд демо үзүүлнэ (жишээ нь код өөрчилж push хийхэд CI автоматаар ажиллах).
4	Асуулт хариулт	Танилцуулга бүрийн дараа багш, ангийнхан 2-3 асуулт асууж, баг хариулна.
5	Харилцан үнэлгээ	Ангийнхан бусад багийн ажилд peer evaluation маягт бөглөнө (тодорхой шалгууруар).
6	Эцсийн feedback	Багш баг бүрд эцсийн товч feedback өгнө.

Бие дааж хийх даалгавар: Демо танилцуулгын бэлтгэл

Багаараа: (1) 5-7 минутын танилцуулгын слайд/script бэлд, (2) амьд демо хийх дараалал (жишээ нь: код өөрчлөх -> push -> CI ажиллах -> deploy) төлөвлө, (3) магадгүй гарах 3 асуултад (жишээ нь "яагаад энэ архитектурыг сонгосон бэ", "AI хэрхэн ашигласан бэ") бэлэн хариулт бэлд.

Хүлээлгэж өгөх зүйл (deliverable): *Танилцуулгын слайд/тэмдэглэл + демо төлөвлөгөө.*